13.1 Gestão de usuários — status de entrega

Bloco 13.1 Gestão de usuários · Sprint 13 — Administração, qualidade e segurança · Data: 2026-10-05 · Branch: `main` · Ambiente: gc-dev (`gzbmhgnpoehormnidmgg`)

Resumo

Validado: a rodada do bloco (`caffeinate -i npm run validar` , 2026-10-06) passa nas sete camadas. Os números estão na seção 4.

Antes, criar usuário exigia mexer direto no Supabase. Agora o admin convida pela tela **Configurações → Usuários**, já com o perfil. O sistema gera um **link de convite** que o admin manda por WhatsApp ou como preferir. A pessoa abre o link, cria a senha e entra. A tela também edita, gera novo link de acesso (convite vencido ou senha esquecida), desativa e reativa.

Ao desenhar a tela, apareceram **duas falhas de segurança** no banco, as duas provadas em gc-dev e corrigidas na **migration nova** `20261005120000_usuarios_protecao_perfil.sql` , aplicada em gc-dev (seção 2).

1. As seis atividades do bloco

ATIVIDADE DO CLICKUP	ONDE FICOU
Tela /configuracoes/usuarios listando os profiles da empresa	`configuracoes/usuarios/page.tsx` e `usuarios-panel.tsx` , mais a aba **Usuários** em Configurações. Cada linha mostra nome, e-mail, telefone, perfil, situação (**Ativo**, **Convite pendente** ou **Inativo**, por `situacaoDoUsuario`) e último acesso. A lista vem de `profiles` sob RLS. O último acesso e a confirmação vêm da API de administração do Auth
Convidar usuário por email (Supabase Auth invite) já com perfil definido	**Decisão do Breno: link de convite, sem e-mail.** O gc-dev usa o SMTP padrão do Supabase, que manda 2 e-mails por hora e só entrega para a equipe da organização. `convidarUsuario` cria o usuário no Auth com `generateLink({ type: 'invite' })` , que não manda e-mail, e o perfil na empresa do admin, com o perfil escolhido. Devolve o link para **Copiar** ou **Enviar pelo WhatsApp** (abre a conversa quando há telefone). O link aponta para a rota `/convite` , que troca o token pela sessão (`verifyOtp`) e leva a `/convite/senha` , onde a pessoa cria a senha (mínimo de 8 caracteres, com confirmação). Se o perfil não grava, o usuário do Auth é desfeito
Editar nome, telefone e perfil	`atualizarUsuario` . O e-mail é o login e não muda pela tela
Ativar e desativar usuário (campo ativo)	`definirAtivo` . Desativar marca `profiles.ativo = false` e **bloqueia no Auth** (`ban_duration`). O admin não tem botão para desativar o próprio acesso, e a action também recusa

ATIVIDADE DO CLICKUP	ONDE FICOU
Bloquear login quando profiles.ativo=false	Em quatro camadas. O **login** é recusado pelo Auth, com o recado "Seu acesso está desativado". A **sessão que estava aberta** cai na próxima navegação: o middleware vê o bloqueio e passa por `/sair`, que limpa os cookies. O **layout** também manda o inativo para `/sair`. E o **RLS** deixa de liberar qualquer dado ao inativo, mesmo com o token ainda válido (seção 2)
Só admin acessa	Duas camadas. O layout de `/configuracoes` (`requirePerfil(['admin'])`) protege a rota, e cada action repete a checagem (admin **e** ativo). No banco, o trigger novo impede quem não é admin de mudar perfil, situação ou e-mail

Gerar link de acesso (o ícone da chave na linha) gera um link `type=recovery`. Serve para o convite vencido e para a senha esquecida, e leva à mesma tela de definir senha. O link vale **1 hora** (`mailer_otp_exp = 3600` no gc-dev) e só funciona uma vez.

2. As duas falhas de segurança corrigidas

Ambas foram provadas em gc-dev em 2026-10-05, numa transação desfeita no fim, antes da correção.

1. Qualquer usuário virava admin pela API do Supabase. A policy "Usuário edita próprio perfil" (desde a migration inicial) deixava o usuário atualizar a própria linha conferindo só o `id` e a empresa, sem travar nenhuma coluna. Com a sessão do comercial, `update profiles set perfil = 'admin' where id = auth.uid()` passava, e `has_perfil(array['admin'])` respondia verdadeiro.

- **Correção:** o trigger `profiles_proteger` (BEFORE UPDATE OR DELETE) deixa só o admin mudar perfil, situação e e-mail, e ninguém muda a empresa de um usuário.
- **A mesma trava protege a operação:** a empresa nunca fica sem admin ativo. Rebaixar, desativar ou excluir o último admin é recusado.

2. Usuário desativado continuava lendo e gravando. As funções `current_empresa_id()`, `current_perfil()` e `has_perfil()`, que todas as policies usam, não olhavam `profiles.ativo`.

- **Correção:** as três passam a exigir o usuário ativo, e o inativo não passa em nenhuma policy.
- **Uma policy nova** deixa todo usuário ler a **própria** linha, para a aplicação conseguir dizer "acesso desativado".

Provas depois da migration, também em transação desfeita:

CASO	RESULTADO
Comercial muda o próprio perfil para admin	**recusado**: "Só o admin muda o perfil, a situação ou o e-mail de um usuário"
Comercial muda o próprio nome	permitido
Admin muda o perfil do comercial	permitido

CASO	RESULTADO
Comercial desativado: `current_empresa_id()`	nulo
Comercial desativado: obras visíveis	0
Comercial desativado: a própria linha	1 (lê)
Rebaixar os 2 admins ativos da empresa	**recusado**: "A empresa precisa de ao menos um admin ativo"

As mesmas regras ficaram permanentes na camada escrita (seção 4).

Registro para quem for operar gc-prod: a migration `20261005120000` corrige falhas que também existem em gc-prod (as policies vêm da migration inicial). Ela precisa ir junto com as pendentes de prod, e antes do go-live (sprint 14). A gestão de usuários precisa de `SUPABASE_SERVICE_ROLE_KEY` no ambiente do servidor; sem ela, a tela avisa e não convida.

3. Achado para o 13.3

O **cadastro aberto** está ligado no Auth do gc-dev (`disable_signup = false`). Qualquer um com a chave pública consegue criar conta pela API. Sem perfil, essa conta não lê nada (as funções das policies devolvem nulo), mas a conta existe. Desligar o cadastro é decisão de configuração do projeto e entra na revisão de RLS e segurança do 13.3.

4. Validação

A rodada final (`caffeinate -i npm run validar`) é de 2026-10-06:

CAMADA	RESULTADO	DO 13.1
sobras	nenhuma em gc-dev	—
estático	**limpo** (`tsc` + `next lint`)	—
unitário	**379 casos**	**6 casos** (`usuarios.test.ts` e `redirecionar.test.ts`)
build	**64 rotas**, iguais à baseline	**4 rotas novas**: `/configuracoes/usuarios`, `/convite`, `/convite/senha` e `/sair`
runtime	**296/296 rotas**	**6 rotas**: `/configuracoes/usuarios` como admin, comercial, financeiro e visualizador (os três últimos redirecionados), `/convite` e `/convite/senha`
dados	**57/57 checagens**	**1 checagem**: a lista da página traz só perfis da empresa (1 registro)

CAMADA	RESULTADO	DO 13.1
escrita	**495/495 passos**	**20 passos**: convite (zod, perfil, guard do comercial, link, e-mail repetido), link de uso único, senha, edição, trigger contra autopromoção, desativação com bloqueio no Auth e RLS, reativação, link de acesso (`recovery`) e limpeza
navegador	**254/254 passos**	**passo 52**, 5 checagens: zod do convite, link e atalho do WhatsApp, link abre a tela de senha com o nome, zod da senha, entrada no dashboard com o perfil escolhido. Screenshots `52-convite-link.png` e `52-convite-senha.png`

Rodadas do bloco:

1. Parou no runtime, em `/convite/senha`.
2. e 3. Pararam no navegador, esperando a tela de senha.
3. Parou no mesmo ponto: o link caía em `/login?tipo=invite`. O redirect de `/convite` saía do host do pedido (`localhost` em vez de `127.0.0.1`) e o cookie da sessão ficava para trás. A correção é `urlNoMesmoHost` (`src/lib/redirecionar.ts`).
4. Ficou 2h53 na escrita porque o Mac dormiu durante a rodada. Parou no 7.2 e deixou uma obra `VALIDA-EXE-*` com item. O `verificar-sobras.mjs` não apagava a obra por causa do item preso a ela. Agora apaga os itens da obra antes.
5. **Final**: passou nas sete camadas com os números da tabela.

5. Pendências

- **E-mail de convite**: fica de fora por decisão. Quando houver um SMTP próprio (Resend, por exemplo), o mesmo convite pode também mandar o e-mail.
- **Validade do link**: 1 hora no gc-dev. Para o uso real, vale avaliar 24 horas (`mailer_otp_exp`), que é configuração do projeto.
- **Conferência no celular**: a cargo do Breno.